



Lecture 17

Observability

Sharif University of Technology
Winter 2026



Learning Objectives

Explain why traditional monitoring techniques break down in a microservice architecture

Explain the difference between monitoring and observability

Describe the core building blocks of observability: log aggregation, metrics, and distributed tracing

Define and calculate SLAs, SLOs, SLIs, and error budgets

Design alerts so they don't create too many unnecessary notifications

Identify testing-in-production techniques and when to use each one

Why Microservices Make This Hard



A microservice architecture breaks one application into many small, independently deployable services that communicate over a network.

The Benefits (the reason we did this)

- Independent deployability
- Technology diversity
- Team autonomy & ownership
- Targeted scaling

The Cost (today's topic)

- More processes to watch
- More network calls that can fail
- More log files, more hosts
- Harder to see the “whole picture”

It's Friday at 4:55pm...



1

The plan

The team is heading out for some fun to start the weekend.

2

The emails arrive

"The website is misbehaving!"

3

The escalation

Social media is unhappy. Your boss is on the phone.

4

The question

Where do we even start looking?

Every outage is like a mystery



“

We replaced our monolith with microservices so that every outage could be more like a murder mystery.

Why it resonates: in a single system, issues like slow performance, strange errors, or high CPU usually come from one clear cause. In many services, there are many possible causes, making it harder to find the real problem.

Finding the cause: before vs. now



Single-Process Monolith

- Website slow? It's the monolith.
- Errors? It's the monolith.
- High CPU? It's the monolith.
- One place to check first — one system causing everything makes debugging easier (even if it's still painful).

Microservice Architecture

- Multiple servers, multiple log files, multiple network hops
- More parts means more chances for failure.
- We need to combine small signals into one clear view.
- We also need tools to explore the data and a better way to think about system health.



Part I

How Monitoring Complexity Grows

We'll walk through three stages of scale!

Single Microservice, Single Host



What should we look for here?

Host metrics

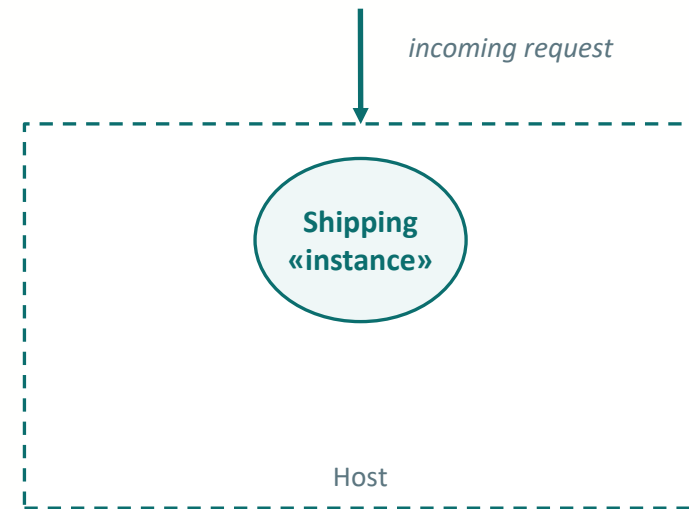
CPU and memory from the machine itself.

Local logs

If a user reports an error, we should be able to find it in the logfile.

Outside-in checks

Response time, or a simple health-check endpoint.



Getting Started Is Easy



At this scale, basic tools seem enough — and that’s the trap. Teams get used to methods that won’t work as the system grows.



Host-level metrics

CPU, memory, and disk are useful, but they don’t show if the system is working correctly



Local logging

Just log to the local filesystem; read it with the command line.



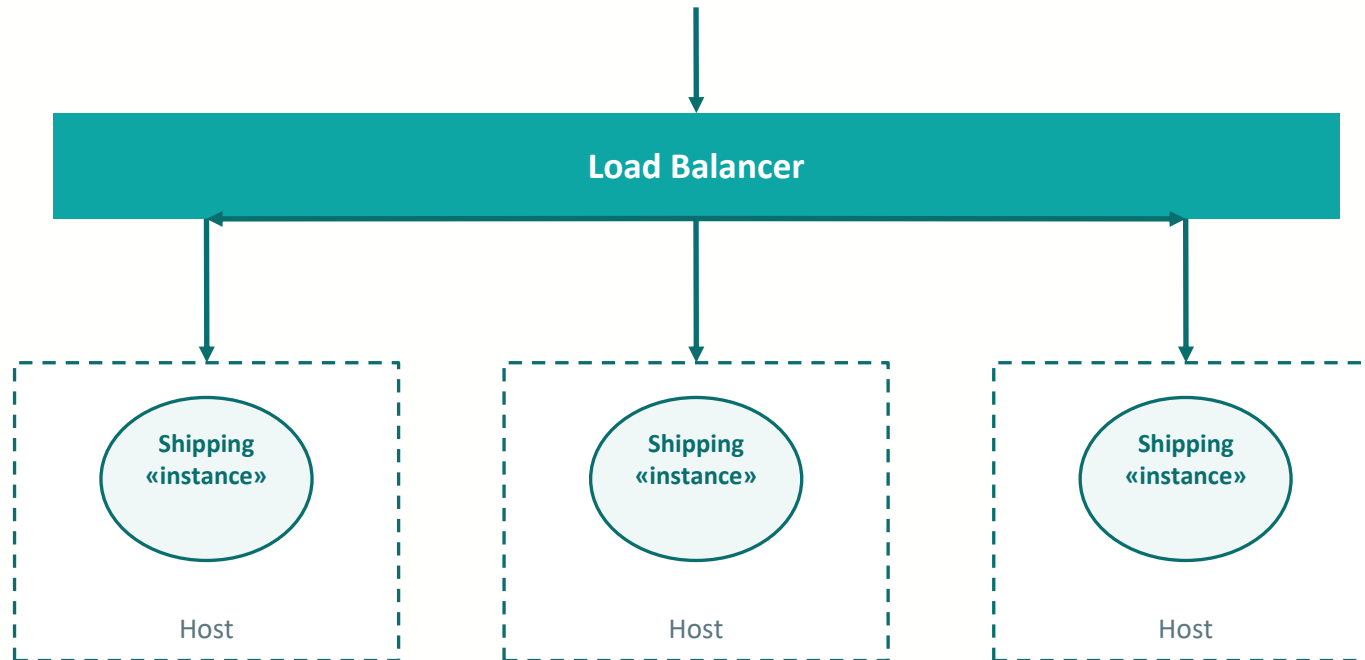
Health checks

A simple endpoint that reports “up” / “down” to a load balancer or operator.

Key takeaway

“Time passes, loads increase, and we find ourselves needing to scale...” — and that's where the easy answers stop working.

Single Microservice, Multiple Hosts



New question: Is high CPU caused by the service code on all machines, or just one bad host where a process is misbehaving?

Aggregate Up, Drill Down



We still want host-level metrics — but now we need to see them aggregated across all hosts and per individual host.

Logs: the SSH-multiplexe

- Tools like SSH multiplexers run the same command on many hosts at once
- `grep "Error"` on a big screen across all hosts can find the culprit
- *It's "good enough" for a while — but it gets old fast as host count grows*

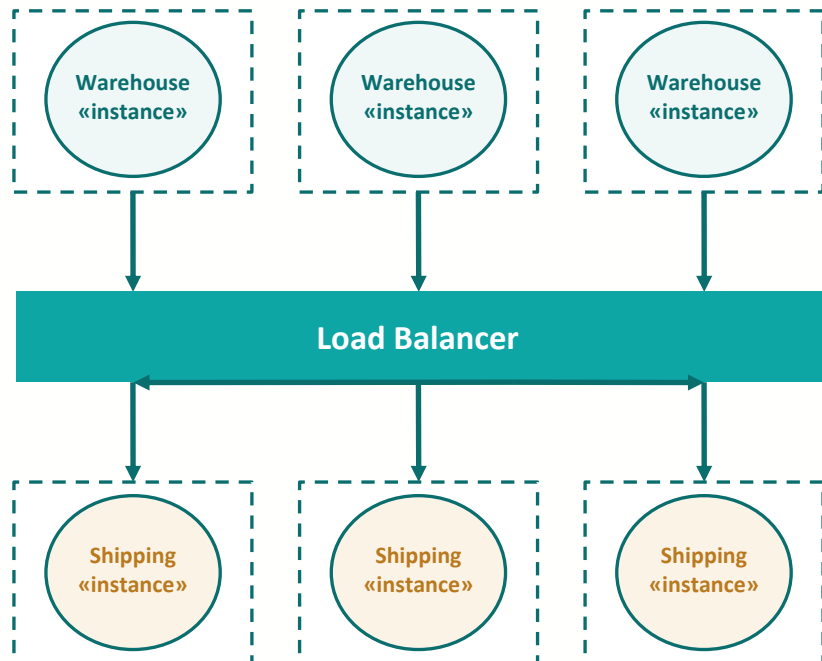
Response time: where do you measure?

- Capture response time at the load balancer for downstream calls
- But what if the load balancer itself is the bottleneck?
- May need to measure at both the load balancer AND each microservice
- Now "healthy" is what matters most — the load balancer removes bad nodes

Multiple Services, Multiple Hosts

Now ask yourself...

- How do you find one error in thousands of log lines, on multiple hosts?
- Is it one bad server, or a system-wide issue?
- How do you trace a deep error back to its root cause?



Aggregation Alone Isn't Enough



Collecting metrics and logs is needed, but not enough. We also need to understand a constant flow of changing data. That needs a new way of thinking, not just more dashboards

A Fairly Static Landscape of Monitoring

Watching dashboards we pre-defined for problems we anticipated



The Active World of Observability

Asking new questions of the system — even ones we didn't plan for

The Three Stages, Compared



Concern	Single Host	Multiple Hosts	Multiple Services
Logs	Read locally	SSH-multiplex + grep	Centralized aggregation
Metrics	Just this host	Aggregate + drill down	Aggregate across services
Response time	One measurement	LB + service level	End-to-end across hops
“What’s broken?”	Obvious	Host vs. service?	Which of N services, and why?

Each jump in scale makes previous tools “good enough” stop working. That’s why we need a new mindset — observability — not just bigger tools



Part II

Observability vs. Monitoring

Two words that get used interchangeably — and shouldn't be.

Two Different Kinds of Thing



Observability

A PROPERTY OF THE SYSTEM

Can you answer questions you didn't think to ask in advance?

EXAMPLE

Checkout is slow for one customer segment, in one region, on one browser. No dashboard exists for that combination — but you can still trace it, because the data needed to explain any internal state is already being captured.



Monitoring

AN ACTIVITY WE DO

Did the specific thing you set up an alert for happen?

EXAMPLE

You set an alert: "page if CPU > 90%." It fires — great, it caught what it was built to catch. But the real issue was a slow database query, and no alert was watching for that.

The difference: monitoring tells you **something is wrong**. Observability lets you figure out **why** — for problems you never anticipated.

Known Unknowns vs. Unknown Unknowns



Traditional Monitoring

- Think in advance about what could go wrong
- Define alerts/thresholds for those scenarios
- Works well... until something you never anticipated happens
- More distributed systems = more things you'd never have thought to check

A Highly Observable System

- Gives you a rich collection of external outputs
- You can explore those outputs in many different ways
- The concrete payoff: you can ask questions of production you never thought to ask before
- Monitoring = the activity; observability = what makes that activity possible at all

The “Three Pillars” of Observability



Metrics

Numeric measurements
over time



Logs

Discrete, timestamped
events



Distributed Tracing

The path of a request across
services

Proposed “MELT” — Metrics, Events, Logs, and Traces — a term that hasn't really caught on.

Looks good at first — simple, easy to remember, and easy to explain. But Sam Newman (and others) say this idea is too simple. Let's see why!

Why “Pillars” Can Mislead



Implementation vs. outcome

Observability is something your system has, not a list of tools you buy. Thinking of it as a shopping list gets it backwards — like using microservices without understanding why you need them

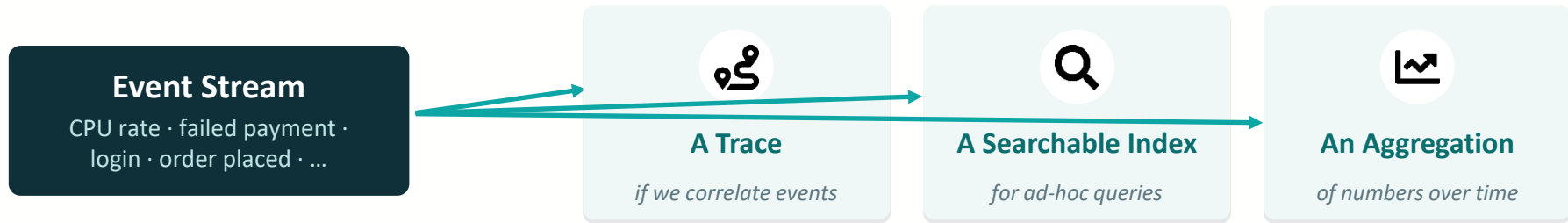
The lines blur

Metrics, logs, and traces overlap in practice: a log line can carry a metric, and a trace can be built from a string of correlated log entries.

It's a great way to sell tools

“You need one tool for metrics, another for logs, another for traces” is an easy product pitch — it doesn't mean it's the right architecture for your needs.

Fundamentally, It's All Just Events



Today's toolchains push us toward collecting these differently — but that shouldn't limit how we think about getting the information we actually need.



Part III

The Building Blocks of Observability

The Building Blocks of Observability



We'll spend most of today on the first three — they're the foundation everything else builds on.



Log
Aggregation



Metrics
Aggregation



Distributed
Tracing



SLA / SLO /
SLI



Alerting



Semantic
Monitoring



Testing in
Production

Log Aggregation: The Problem



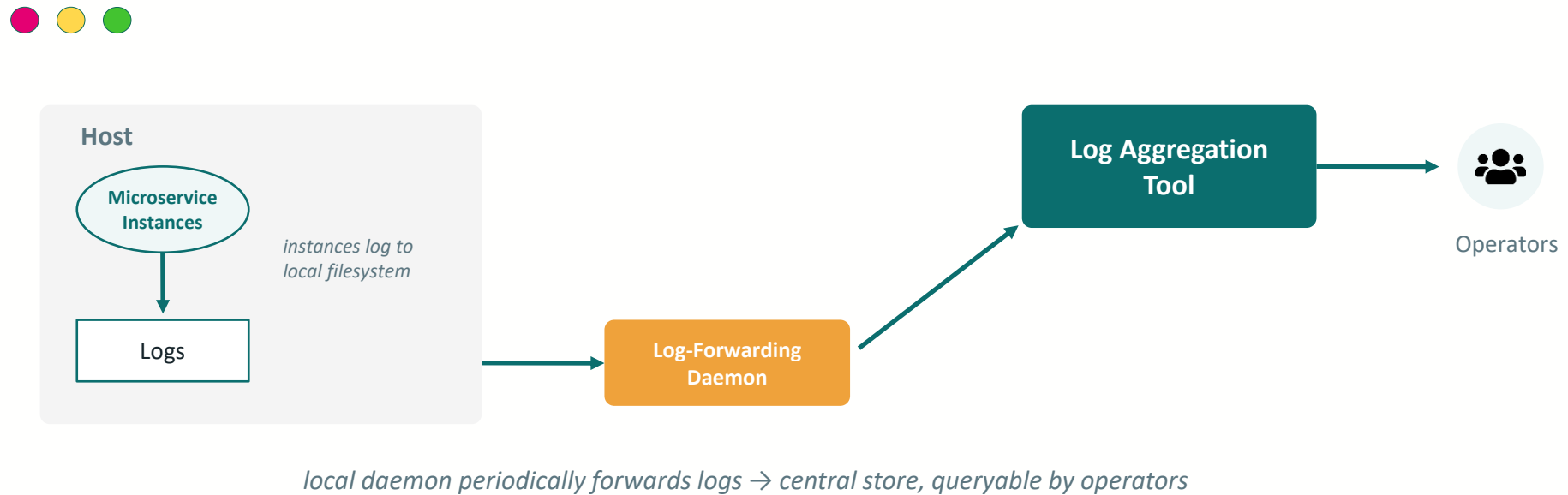
- With many servers and service instances, SSHing into each box stops scaling. We need a system that **collects logs centrally** and makes them **queryable**.
- Logs become the most vital tool for understanding what's happening in production
- They help you diagnose what already went wrong AND tell you something needs attention right now
- In simple monolith systems, logging is often added later. But in distributed systems, you can't treat it as an afterthought.



"Before Anything Else"

Get a log aggregation tool up and running before you do anything else to build out your microservice architecture. Consider it a prerequisite, not an afterthought.

Log Aggregation Architecture



Why this works well: your microservice doesn't need to know about the logging system. It just writes logs to a local file, with no extra APIs to call. But you still need to understand the risks in how logs are shipped — for example, logs can be lost while being sent.

A Common Log Format Matters



To search across combined logs, you need date, time, service name, and log level in consistent places. Otherwise, your queries become hard or even impossible to write



Reformat at the source

It's better to change the log format in the microservice itself, instead of fixing it later in the log shipping layer. Doing reformatting on the way can waste a lot of CPU when traffic is high.



No universal standard

Most teams take the classic web server access log format and extend it with extra fields. The key is to pick one format and standardize it everywhere internally.



Structured (JSON) logs

Useful for finding a field like a customer ID reliably — at the cost of being harder for a human to read directly.

Reading a Simple Log Format



```
15-02-2020 16:00:58 Order INFO [abc-123] Customer 2112 has placed order 988827  
15-02-2020 16:01:01 Payment INFO [abc-123] Payment $20.99 for 988827 by cust 2112
```

Each value sits in a fixed column: date, time, service name, level, then message.

Works well when...

- You're querying by date, service name, or level
— these sit in static, predictable columns

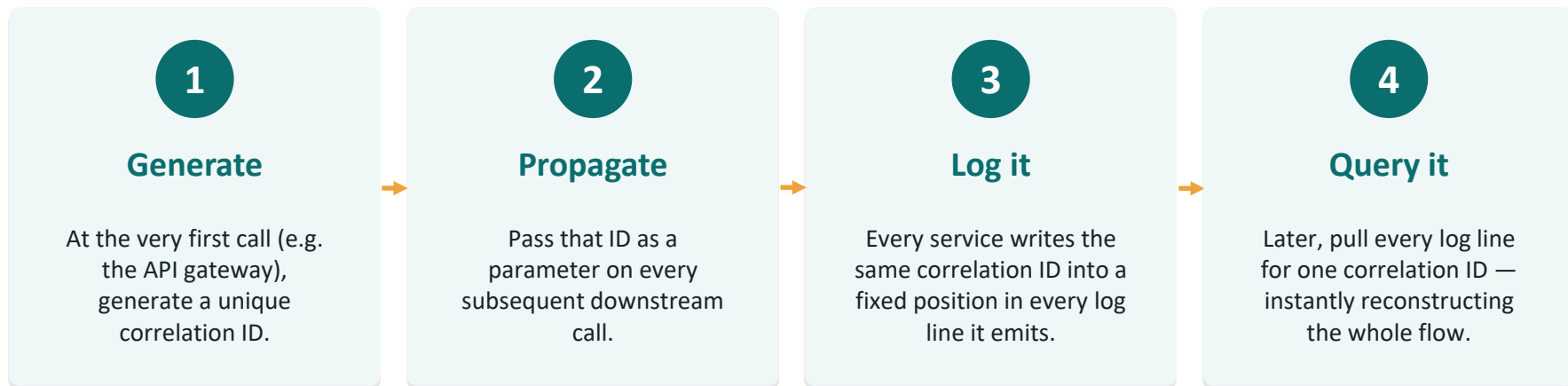
Breaks down when...

- You want to find all log lines for one customer
— the customer ID sits in a different position in each line

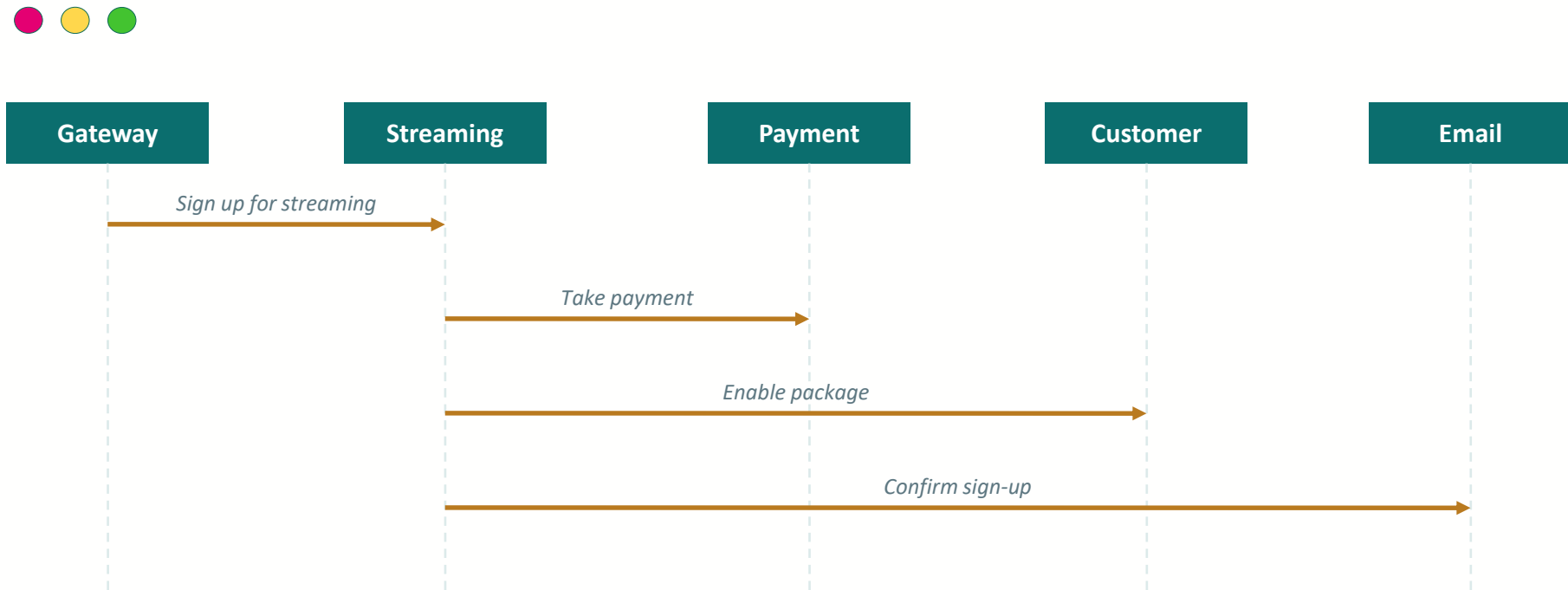
Correlation IDs: Connecting the Dots



A single user action can trigger calls across five, ten, or fifty services. How do you group all the log lines from one logical request together?



Tracking a Streaming Sign-Up



Correlation ID abc-123 generated once at the Gateway, then passed unchanged to every downstream call

Correlation IDs in the Logfile



```
15-02-2020 16:01:01 Gateway INFO [abc-123] Signup for streaming
15-02-2020 16:01:02 Streaming INFO [abc-123] Cust 773 signs up ...
15-02-2020 16:01:03 Customer INFO [abc-123] Streaming package added ...
15-02-2020 16:01:03 Email INFO [abc-123] Send streaming welcome ...
15-02-2020 16:01:03 Payment ERROR [abc-123] ValidatePayment ...
```

Without a correlation ID, a “Payment error” is just a random, unclear event. With it, one query (“find everything tagged abc-123”) can rebuild the full story — sign-up, plan change, welcome email attempt, and the exact point of failure

It's hard to add later — so implement correlation IDs from day one

Can We Trust the Timestamps?



Each log line carries a timestamp generated on the machine that wrote it. But machine clocks drift — even a one-second skew can completely scramble your understanding of “what happened first.”

- **Clock skew is everywhere**

Even with the Network Time Protocol (NTP) running, skew is reduced — never fully eliminated.

- **Two real limitations**

(1) We can't get fully accurate timing for the whole request flow. (2) We can't reliably rebuild what caused what.

- **A partial fix: logical clocks**

Logical clocks use a counter (not wall-clock time) to create a reliable ordering of events

- **The real fix: distributed tracing**

A dedicated tracing tool solves both problems — it gives you accurate timing across the whole request flow and shows how each step connects to the next

Shortcomings of Logs



Ordering & timing

Clock skew undermines both the order of events and accurate latency measurement.



Volume & cost

More services and calls = huge data volumes. Indexing costs and storage bills grow with it.



Sensitive data

Logs often contain valuable, sensitive information — they become both an access-control problem and an attack target.

One real story: a team's largest Elasticsearch cluster could only store about six weeks of logs for a single product — which meant constant data movement just to keep things running. Plan retention carefully. Don't assume "more logging" is free.

Log Aggregation: Tooling Landscape



Category	Examples	Notes
Open source stack	Fluentd → Elasticsearch → Kibana	Popular, but Elasticsearch ops overhead is real
Open-core alternative	Humio	Designed for high-scale ingestion
Commercial SaaS	Datadog, Splunk	Powerful, but watch the bill at scale
Cloud-native	AWS CloudWatch, Azure App Insights	Good enough out-of-the-box, but locks you into one cloud

A reminder: log aggregation platforms are themselves production systems — mission critical, and a potential attack surface (see the 2020 SolarWinds supply-chain breach). Treat them with the same diligence as your application code.

Metrics Aggregation



Our website is seeing 50 4XX errors per second — is that bad? Catalog service CPU is up 20% since lunch — should we panic? You can only answer these questions if you have enough history of what “normal” looks like.

- New instances come and go frequently, so collection should be easy to set up on any new host
- You need a metric at multiple levels: the whole system, one service, or one single instance
- That requires attaching metadata (tags) to each metric so you can slice by service, host, version...
- Metrics are also key for capacity planning — when infrastructure is on demand, understanding your real usage patterns really matters

Resolution vs. Retention



Right Now

1 sample / 10 seconds
for the last 30 minutes

High resolution, so you can react to what's unfolding right now.

Last Month

1 average / hour

Good for overall trend analysis — it keeps storage and query costs manageable, but aggregation removes detail. You have to decide upfront what detail you're willing to lose

Cardinality: How Many Ways Can You Slice It?



Cardinality is the number of unique label combinations in your data. The more ways you can slice and query the data, the higher the cardinality

Low Cardinality

- e.g. CPU rate for a single machine
- Cheap to store, fast to query
- What classic systems like Prometheus were built for

High Cardinality

- e.g. service + customer ID + request ID + build number + product ID, all queryable together
- Lets you ask questions you never anticipated
- Needs purpose-built storage (e.g. Honeycomb, Lightstep)

The Storage Engine Pays the Price



Storing a metric is very cheap, but storing tags is expensive. If you attach too many tags to each metric, your storage system can quickly slow down or break

— Charity Majors, founder of Honeycomb

From the Prometheus documentation:

Don't use labels for high-cardinality data — things like user IDs, email addresses, or any unbounded set of values where there are too many possible combinations

Takeaway: systems designed for low cardinality are very hard to adapt for high cardinality later. Choose the right tool upfront, based on the kinds of questions you expect to ask

Metrics Tooling at a Glance



Tool	Cardinality	Best for
Graphite	Low	Classic time-series trends (legacy default)
Prometheus	Low – Medium	Pull-based metrics, k8s-native ecosystems
Honeycomb	High	Ad-hoc, exploratory questions in production
Lightstep	High	Tracing-centric, high-cardinality analysis

Important reminder

Your monitoring and observability tools are production systems themselves. They need the same operational discipline — backups, access control, and on-call ownership — as the applications they monitor.

Distributed Tracing: Beyond Correlation IDs



A correlation ID is a good starting point, but it's still a fairly simple approach. To truly understand relationships between calls — and see where time is actually spent — you need purpose-built distributed tracing

- A microservice architecture is, fundamentally, a set of processes collaborating to complete a task
- Understanding production behavior means seeing the relationships between those processes
- This helps you see how big the problem is and exactly which step is causing it.
- Correlation IDs in logfiles **work**, but you end up building **custom tooling** just to visualize them — **distributed tracing tools** do this out of the box

Spans → Traces



One checkout request, three spans, one trace

Span 1

API Gateway

Duration: 0–12ms

Tags: request_id, route



Span 2

Auth Service

Duration: 12–18ms

Tags: customer_id, auth_result



Span 3

Payment Service

Duration: 18–47ms

Tags: order_id, amount

Local activity within one thread or service call is captured as a span — each one timestamped and tagged.



A central collector correlates all three spans — via a shared *trace ID* — into one TRACE: the full 47ms checkout journey.

With the **OpenTelemetry** API each span carries a start/end time, associated logs, and an arbitrary set of key-value tags — customer ID, order ID, hostname, build number, and more.

How a Distributed Trace Works



Reading this trace: the API call spends most of its 580ms in `ticketBackend.fetchTicketsForExport`, which is mainly caused by seven MySQL queries running one after another. The fraud check happens in parallel near the end. You can't see this from plain logs — you need a trace for this kind of insight.

Sampling: Not Every Span Survives



Capturing trace data for every request can noticeably slow your system. That's why tracing systems use sampling — they intentionally drop some data to keep the system fast and healthy.

Basic random sampling

Google's Dapper used highly aggressive random sampling. Jaeger's default captures only 1 in 1,000 calls.

Dynamic / smart sampling

Honeycomb and Lightstep use adaptive sampling — for example, they keep every error, but only sample 1 in 100 identical successful requests

The goal is to collect enough data to understand the system, without collecting so much that data collection becomes a bottleneck itself

Implementing Distributed Tracing

**1**

Instrument

Capture span data inside your microservices. Libraries built on OpenTracing/OpenTelemetry often automatically instrument common calls like HTTP and database access — but you'll still need to instrument your own business logic

**2**

Forward

Send span data to a collector — usually through a local forwarding agent, just like log aggregation. The agent can also adjust sampling or add extra tags

**3**

Collect

A central collector receives spans and reconstructs them into traces you can query, filter, and visualize.



Part IV

Are We Doing OK?

SLA — Service-Level Agreement



An agreement between the people building a system and the people using it: what users can expect, and what happens if the system fails to deliver it.

Real Example: AWS EC2 Compute SLA

- AWS does not guarantee uptime for a single EC2 instance — there's no individual-host availability promise at all
- Its “best efforts” target is around 90% uptime for any given instance
- If that target is missed, AWS simply doesn't charge you for that hour — not a service credit, just a billing adjustment
- In practice, real-world AWS uptime is far higher than the SLA promises — SLAs tend to describe the bare legal minimum, not the expected experience

SLO — Service-Level Objective



An SLA is broad and organization-wide. An SLO is what a single team commits to deliver — the practical, team-level version of that promise



Examples

Expected uptime, or an acceptable response-time target for a specific operation.



Not just SLA math

If every team meets its SLOs, the company will easily meet its SLAs. But SLOs are often set as internal goals too, not just strict guarantees



Team ownership

SLOs often reflect goals the team itself wants — they don't have to map 1:1 to any external promise.

SLI — Service-Level Indicator



To know whether you're meeting an SLO, you need real data. That measurement is the SLI.

SLIs are clear, measurable indicators of how a system is actually performing:

- A process's response time
- A successful customer registration
- An error shown to a customer
- An order successfully placed

The flow is: collect SLIs → compare them against SLOs → roll them up to determine whether the organization is meeting its SLA. Each step moves from raw measurements, to defined targets, and finally to a formal promise

How SLA, SLO, and SLI Relate



SLI

Measurement

"99.95% of requests returned in <300ms last week"



SLO

Team Target

"We aim for 99.9% of requests under 300ms, per quarter"



SLA

Org. Promise

"We guarantee 99.5% uptime. If we fall below that, you receive a credit as compensation"

Error Budgets: Permission to Take Risks



Trying new things always adds some instability. If we only focus on stability, we end up blocking change altogether. An error budget makes it clear how much failure is acceptable

Worked calculation

SLO: 99.9% availability per quarter, 24/7

→ **Allowed downtime per quarter = 0.1% of ~13 weeks = 2 hours, 11 minutes**

That 2h11m is your error budget for the quarter.

Decision rule: if you are well under budget, you can confidently ship a risky new release. If you are already over budget, pause new changes and focus on improving reliability instead

Using Error Budgets to Guide Team Behavior



Give teams breathing room

Knowing exactly how much risk is “in the bank” frees teams to experiment without guessing.



Align incentives

Reliability and feature velocity stop being opposing goals — they're both visible on the same dashboard.



Make trade-offs explicit

“Should we ship this risky change now?” becomes a data-informed conversation, not a gut call.

This idea comes from Site Reliability Engineering (SRE), a practice pioneered at Google. It focuses on keeping systems reliable while still enabling continuous change and improvement.

Some Problems Are Worse Than Others



The key question for any alert is: “Should this wake someone up at 3 a.m.?” As the number of possible failures grows, prioritizing which alerts truly matter becomes critical



True Story: Google's Velcro Hard Drives

During a visit to Google’s Mountain View campus, an old server rack on display had no enclosures—just bare motherboards, with hard drives held in place using Velcro instead of screws

“The hard drives fail so often that we don’t bother screwing them in. We just pull them out, toss them, and Velcro in a new one.”

The system was built with the assumption that hardware will fail. A single failed drive wasn’t an incident — it was normal maintenance, not a 3 a.m. page. Each day, someone would walk the racks and simply replace any failed drives as part of routine work.



Part V

Semantic Monitoring

Semantic Monitoring: What Does “Correct” Mean?



Instead of simply looking for errors, semantic monitoring asks a different question: is the system behaving as expected in terms of real business outcomes?

MusicCorp's model for “correctly behaving”:

- New customers can successfully register to join
- We are selling at least \$20,000 worth of products per hour during peak traffic.
- Orders are still shipping at a normal rate, with no noticeable slowdown or backlog.

These are high-level value statements, not low-level thresholds like “disk usage below 95%.” Defining them usually requires collaboration with the product owner — it’s not a decision operators make on their own

Real User Monitoring vs. Synthetic Transactions



Real User Monitoring

- Watch what's actually happening in production right now
- e.g. count real sign-ups, real orders shipped, real revenue
- Pro: reflects ground truth
- Con: noisy, and only tells you AFTER the problem already affected a real customer

Synthetic Transactions

- Injecting synthetic user behavior with known inputs and expected outputs—for example, creating a fake customer and verifying it was created correctly
- Pro: catches problems before a real user does, with low noise
- Con: requires careful test-data management to avoid polluting production

Testing-in-Production Techniques



Technique	What it does
A/B testing	Two versions live at once; users see one of each, compared on outcome
Canary release	New version rolled out to a small slice of users first, then expanded
Parallel run	Old and new versions both execute on every request; user sees only one
Smoke test	Quick automated checks right after deploy, before full release
Synthetic transaction	A full, fake end-user interaction injected on a schedule
Chaos engineering	Deliberately injecting faults (e.g. killing a VM) to test resilience

All of these are, in a sense, monitoring activities—they check that your production system is behaving as expected and often catch issues before users notice them



Part VI

Wrap-Up

Standardization



Microservices give teams freedom to choose their own tech — but monitoring and observability is one of the few areas where standardizing across the whole organization pays off enormously.

- Standardize your log format — so queries work the same way everywhere
- Put all metrics in one place, with standard names (ResponseTime, not RspTimeSecs in one service and ResponseTime in another)
- Make it easy to do the right thing: a shared platform with log aggregation and metrics built in beats asking every team to build its own
- This increasingly becomes the job of a dedicated platform team

5 Criteria for Selecting Observability Tools



Democratic

Usable by everyone on the team, not just senior operators — and affordable enough to use in dev/test, not only production.



Easy to Integrate

Supports open standards (OpenTelemetry) so instrumentation and vendor migration stay cheap.



Provides Context

Shows you how a signal compares to history, to other signals, to its dependencies, and to its overall impact.



Real-Time

Seconds, not hours — you need information fast enough to beat your users to the discovery.



Suitable for Your Scale

Tools built for Google-scale make extreme trade-offs you probably don't need — and probably can't afford.

Getting Started: A Checklist



- Capture host-level metrics (CPU, I/O) and tie instances back to their host
- Capture response times for every service interface; log downstream calls
- Get correlation IDs into your logs from day one — they're hard to retrofit later
- Stand up a basic metrics + log aggregation toolchain before anything fancier
- Don't feel obligated to start with a dedicated distributed tracing tool — add it once a managed option makes it easy, or complexity demands it
- Build at least one synthetic transaction for a key business operation
- Keep asking: “Am I confident this system is working for my users right now?”

Key Takeaways



Monitoring is an activity; observability is a property of your system that makes that activity worthwhile

Log aggregation + correlation IDs are the highest-value, lowest-effort first step — do this before anything else

Metrics, logs, and traces overlap more than the “three pillars” framing suggests — think in terms of events

System health isn’t just “up” or “down.” Think of it as a whole system, and use SLOs and error budgets to define what “good enough” really means

Design alerts on purpose. Too many alerts can wear people out and lead to serious failures, even outside software

Testing in production (synthetic transactions, canaries, chaos engineering) is not reckless — done well, it's one of the safest ways to build confidence

Textbooks



- Sam Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2021.



Thank You!

Questions?